Adding Custom System Calls to xv6-riscv Operating System

Alireza Doostimehr
Student Number: 400442135

February 2026

1 Introduction

This project adds a custom `getticks` system call to the xv6-riscv operating system. This system call returns the number of timer ticks since the system started running.

The complete source code and implementation details are available at: `https://github.com/alirezadoostimehr/xv6-no-risks`

2 Implementation Process

Adding a system call to xv6-riscv requires changes to several files across both user space and kernel space. The following sections explain each step in detail.

2.1 System Call Number Assignment

Every system call needs a unique identification number. This number is used by the kernel to determine which function to execute when a system call is made.

File: `kernel/syscall.h`

```
1 #define SYS_getticks 23
```

This definition assigns the number 23 to our new system call. The kernel uses this number to look up the correct function in its system call table.

2.2 User-Space Interface

User programs need to know the function signature to call the system call properly.

File: `user/user.h`

```
1 int getticks(void);
```

This declaration tells the compiler that `getticks` is a function that takes no arguments and returns an integer value.

2.3 User-Space Stub Creation

The user-space stub generates the assembly code that performs the actual system call.

File: `user/usys.pl`

```
entry("getticks");
```

This Perl script generates assembly code that loads the system call number into register `a7` and executes the `ecall` instruction to trap into kernel mode.

2.4 Kernel Implementation

The kernel function implements the actual logic of the system call.

File: `kernel/sysproc.c`

```
uint64 sys_getticks(void) {
  uint temp;
  acquire(&tickslock);
  temp = ticks;
  release(&tickslock);
  return temp;
}
```

This function reads the global `ticks` variable, which is updated by the timer interrupt handler. The function uses a spinlock (`tickslock`) to ensure thread-safe access, preventing race conditions when multiple processes might try to read the ticks value at the same time.

2.5 System Call Registration

The kernel needs to know which function to call when it receives a system call request.

File: `kernel/syscall.c`

First, we declare the external function:

```
extern uint64 sys_getticks(void);
```

Then, we add it to the system call table:

```
static uint64 (*syscalls[])(void) = {
  // ... other system calls ...
  [SYS_getticks] sys_getticks,
};
```

This array maps system call numbers to their corresponding kernel functions. When a user program executes `ecall` with system call number 23, the kernel looks up this array and calls `sys_getticks()`.

2.6 Test Program

To verify the system call works correctly, we created a test program.

File: `user/getticks.c`

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
  printf("Now: %d\n", getticks());
  exit(0);
}
```

2.7 Build Configuration

Finally, we need to tell the build system to compile our code.

File: `Makefile`

```
UPROGS=\
  # ... other programs ...
  $U/_getticks\
```

3 Execution Flow

When a user types `getticks` in the xv6 shell and presses enter, the following sequence of events occurs:

1. The shell executes the `getticks` program from the filesystem
2. The program starts running in user mode and calls the `getticks()` function
3. The user-space stub (generated by `usys.pl`) loads syscall number 23 into register `a7`
4. The stub executes the `ecall` instruction, which triggers a trap into kernel mode
5. The CPU switches to supervisor mode and jumps to the trap handler
6. The kernel reads the syscall number from register `a7`
7. The kernel looks up `syscalls[23]` in the system call table and finds `sys_getticks`
8. The kernel calls `sys_getticks()`, which:
   - Acquires the `tickslock` spinlock
   - Reads the current value of the `ticks` global variable
   - Releases the lock
   - Returns the value
9. The kernel puts the return value in register `a0`

10. The CPU returns to user mode
11. The `getticks()` function returns with the value from `a0`
12. The program prints the result using `printf`
13. The program exits

The `ticks` variable is a global counter in the kernel that gets incremented every time a timer interrupt occurs.

4 Testing Results

The `getticks` system call was tested by running the test program at different times after system boot:

```
$ getticks
Now: 271
$ getticks
Now: 329
```

The increasing tick values show that the system call correctly reads the kernel's tick counter.

4.1 Additional Work

A `revrev` system call was also added as a fun exercise. This system call takes a string as input, reverses it, and prints the reversed string.